



Normalização

Aula 12 · Bloco 3 — Modelo Relacional

Por que uma tabela mal desenhada mente — e como consertar

Nesta aula

1. As três anomalias
2. Dependência funcional
3. 1FN, 2FN, 3FN
4. BCNF — a 3FN levada a sério
5. Decomposição sem perda

As três anomalias

Volte à planilha da aula 01. Ela sofre de três males, e todos vêm da mesma causa:

De inserção — não dá para cadastrar uma obra sem inventar um empréstimo falso.

De atualização — trocar o telefone da Ana exige alterar **todas** as linhas dela.

De exclusão — apagar o último empréstimo de um aluno apaga **o aluno junto**.

A causa é sempre a mesma

A tabela está guardando fatos
sobre **mais de uma coisa**.

Alunos e empréstimos misturados
na mesma linha.

Normalizar é **separar os fatos**
até que cada tabela fale de uma coisa só.

Dependência funcional

$A \rightarrow B$ lê-se: "*A determina B*".

Se você conhece A , o valor de B está decidido.

matricula $\rightarrow$ nome



a matrícula decide o nome

nome $\rightarrow$ matricula



nomes repetem

É a ferramenta formal para responder "*este atributo pertence a esta tabela?*".

Os três tipos que interessam

Total — o atributo depende da **chave inteira**. É o que se quer.

Parcial — depende de **parte** de uma chave composta. Quebra a 2FN.

Transitiva — depende de outro atributo **não-chave**, que por sua vez depende da chave. Quebra a 3FN.

```
matricula → cod_curso → nome_curso
           └── transitiva ───┘
```

1FN — valores atômicos

Nenhuma célula guarda lista, nem grupo repetido.

✗ `ALUNO(matricula, nome, telefones)`
 `telefones = "3399-1111, 99999-2222"`

✓ `ALUNO(matricula, nome)`
 `TELEFONE(matricula, numero)`

💡 É exatamente a regra 6 do mapeamento (aula 10). A 1FN não é uma regra nova — é a mesma decisão, vista pelo outro lado.

2FN — sem dependência parcial

Só faz sentido em tabela com **chave composta**.

✗ ITEM(cod_pedido, cod_produto, quantidade, nome_produto)
└ depende só de cod_produto

✓ ITEM(cod_pedido, cod_produto, quantidade)
PRODUTO(cod_produto, nome_produto)

O **nome_produto** não depende do pedido. Ele estava na tabela errada.

3FN — sem dependência transitiva

✗ `ALUNO(matricula, nome, cod_curso, nome_curso)`
└─ nome_curso depende de cod_curso ─┘

✓ `ALUNO(matricula, nome, cod_curso)`
`CURSO(cod_curso, nome_curso)`

Enquanto `nome_curso` viver em `ALUNO`, ele estará repetido em toda linha do curso — e vai divergir.



O resumo que se decora

Todo atributo não-chave depende

da chave (1FN),

da chave inteira (2FN),

e de nada além da chave (3FN).

BCNF — a 3FN levada a sério

A 3FN deixa um caso passar: quando um atributo **não-chave** determina parte de uma **chave candidata**.

A BCNF exige que **todo determinante seja chave candidata** — sem exceção.

💡 Na prática, uma relação em 3FN quase sempre já está em BCNF. Os casos que diferem envolvem múltiplas chaves candidatas sobrepostas, e são raros o bastante para você reconhecer quando aparecerem.

Decomposição sem perda

Dividir uma tabela em duas só é válido se **a junção das duas devolver exatamente a original** — nem linha a menos, nem linha inventada.

A garantia: o **atributo comum** às duas precisa ser **chave** em pelo menos uma delas.



A regra do curso

Toda decomposição entregue vem com a **verificação de perda escrita**:

quais são os atributos comuns,
e de qual relação eles são chave.

Uma decomposição **com perda** é pior
que a relação não normalizada —
porque troca redundância por **mentira**.

E o caminho de volta

Desnormalizar é reintroduzir redundância **de propósito**, por desempenho.

É legítimo — em três condições:

1. A medição mostrou que a junção é o gargalo **de verdade**;
2. Existe um mecanismo que mantém a cópia sincronizada;
3. A decisão está **escrita**, com o motivo.

⚠ Desnormalizar por comodidade, antes de medir, é só o modelo errado com um nome bonito.

Exercícios da aula

Na pasta `aula-12/` :

1. `ex01.md` — as três anomalias numa tabela dada, com exemplos concretos;
2. `ex02.md` — escreva todas as dependências funcionais de um esquema;
3. `ex03.md` — normalize até a 3FN, **justificando cada passo**;
4. `ex04.md` — verifique se uma decomposição dada é sem perda;
5. **Desafio** 🌶️ `ex05.md` — um caso de BCNF que a 3FN deixa passar.

Fim do Bloco 3

O modelo está desenhado, traduzido em tabelas e limpo de redundância.

Bloco 4 — SQL e Projeto Físico

Agora ele vira um banco **rodando**.